

06

Chaves e integridade

Relacionar tabelas e proteger a consistência

AULA

OBJETIVOS DA AULA

- Definir FOREIGN KEY entre tabelas
- Garantir integridade referencial
- Restringir valores com CHECK e DEFAULT
- Entender o papel do InnoDB nas FKs

CRONOGRAMA · 19h – 22h

19:00–19:20	Abertura, chamada e revisão da aula anterior
19:20–20:30	Teoria + demonstrações de código no Workbench
20:30–20:45	Intervalo — 15 minutos
20:45–21:40	Exercícios: leve → médio → alto (mão na massa)
21:40–22:00	Correção comentada (gabaritos) e dúvidas

Chave estrangeira e restrições

- **FOREIGN KEY**
- Liga uma coluna à chave primária de outra tabela.
-  **Regras importantes**
- A PK referenciada precisa existir na tabela-pai.
- Não se pode inserir uma FK inexistente.
- Excluir o pai é bloqueado se houver filhos (salvo ON DELETE CASCADE).
- **InnoDB**
- É a engine do MySQL que realmente aplica as FKs.

```
Restrições na criação

CREATE TABLE funcionarios (
  id      INT AUTO_INCREMENT
          PRIMARY KEY,
  nome    VARCHAR(100),
  cargo   VARCHAR(50),
  situacao VARCHAR(10)
          DEFAULT 'Ativo',
  CHECK (situacao IN
        ('Ativo', 'Inativo'))
);
```

Definindo a chave estrangeira

```
Consulta.sql - MySQL

-- Tabela-pai
CREATE TABLE Restaurantes (
  RestauranteID INT PRIMARY KEY,
  Nome          VARCHAR(100) NOT NULL,
  Categoria     VARCHAR(50)  NOT NULL
);

-- Tabela-filha com FK a nível de tabela
CREATE TABLE Pedidos (
  PedidoID      INT PRIMARY KEY,
  ValorTotal    DECIMAL(10,2) NOT NULL,
  RestauranteID INT,
  FOREIGN KEY (RestauranteID)
    REFERENCES Restaurantes(RestauranteID)
);

-- Caso a FK não tenha sido criada junto:
ALTER TABLE Pedidos
  ADD CONSTRAINT FK_Pedidos_Rest
  FOREIGN KEY (RestauranteID)
  REFERENCES Restaurantes(RestauranteID);
```

COMO FUNCIONA

Nível de tabela

- FOREIGN KEY (col)...

REFERENCES

- aponta a PK do pai.

ALTER + ADD

- cria a FK depois.



EXERCÍCIO PRÁTICO · AULA 6 · NÍVEL LEVE

Turmas e alunos

CONTEXTO

Cada aluno pertence obrigatoriamente a uma turma. Modele isso.

SUA TAREFA

1. Crie a tabela Turmas: turma_id (PK) e nome_turma (VARCHAR 50).
2. Crie a tabela Alunos: aluno_id (PK), nome_aluno (VARCHAR 100) e turma_id (FK para Turmas).
3. Confira as duas estruturas com DESCRIBE.



Turmas e alunos — solução

```
Solucao.sql - MySQL

CREATE TABLE Turmas (
  turma_id    INT PRIMARY KEY,
  nome_turma  VARCHAR(50)
);

CREATE TABLE Alunos (
  aluno_id    INT PRIMARY KEY,
  nome_aluno  VARCHAR(100),
  turma_id    INT,
  FOREIGN KEY (turma_id) REFERENCES Turmas(turma_id)
);
```

PONTOS-CHAVE

- Crie a tabela-pai (Turmas) primeiro.
- A FK usa FOREIGN KEY (col) REFERENCES pai(col).
- InnoDB é a engine padrão que aplica a FK.



EXERCÍCIO PRÁTICO · AULA 6 · NÍVEL MÉDIO

Inserindo com integridade

CONTEXTO

Agora popule as tabelas e teste as regras da chave estrangeira.

SUA TAREFA

1. Insira 3 turmas.
2. Insira 4 alunos, cada um vinculado a uma turma existente.
3. Tente inserir um aluno com turma_id inexistente e observe o erro.
4. Liste os alunos junto do nome da turma (prévia de JOIN).



Inserindo com integridade — solução

```
Solucao.sql - MySQL

INSERT INTO Turmas VALUES
  (1,'ADS'), (2,'Ciência da Computação'), (3,'Gestão de TI');

INSERT INTO Alunos VALUES
  (1,'Marina',1), (2,'Paulo',2), (3,'Rita',1), (4,'Tiago',3);

-- Erro: turma 99 não existe
INSERT INTO Alunos VALUES (5,'Erro',99);

-- Prévia de JOIN
SELECT a.nome_aluno, t.nome_turma
FROM Alunos a
JOIN Turmas t ON a.turma_id = t.turma_id;
```

PONTOS-CHAVE

- Insira Turmas antes de Alunos.
- turma_id do aluno deve existir em Turmas.
- FK inexistente = erro de integridade.



EXERCÍCIO PRÁTICO · AULA 6 · NÍVEL ALTO

Projetos e clientes

CONTEXTO

Uma empresa gerencia Projetos e Clientes. Modele com integridade completa.

SUA TAREFA

1. Crie Clientes: id_cliente (PK, AUTO_INCREMENT), nome (VARCHAR 100) e status com CHECK ('Ativo'/'Inativo') e DEFAULT 'Ativo'.
2. Crie Projetos: id_projeto (PK, AUTO_INCREMENT), nome (VARCHAR 100) e id_cliente como FK para Clientes.
3. Insira 2 clientes e 3 projetos ligados a eles.
4. Tente excluir um cliente que tenha projetos e observe o bloqueio.



Projetos e clientes — solução

Solucao.sql - MySQL

```
CREATE TABLE Clientes (  
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  status VARCHAR(10) DEFAULT 'Ativo',  
  CHECK (status IN ('Ativo','Inativo'))  
);  
  
CREATE TABLE Projetos (  
  id_projeto INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  id_cliente INT,  
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)  
);  
  
INSERT INTO Clientes (nome) VALUES ('ACME'), ('Globex');  
INSERT INTO Projetos (nome, id_cliente) VALUES  
  ('Site', 1), ('App', 1), ('ERP', 2);
```

PONTOS-CHAVE

- CHECK + DEFAULT protegem o status.
- A FK impede apagar um cliente com projetos.
- AUTO_INCREMENT numera as PKs sozinho.

07

JOINS — parte 1

Combinar tabelas com INNER e LEFT JOIN

AULA

OBJETIVOS DA AULA

- Entender por que precisamos de junções
- Usar INNER JOIN (interseção)
- Usar LEFT JOIN (todos da esquerda)
- Ler o resultado de cada junção

CRONOGRAMA · 19h – 22h

19:00–19:20	Abertura, chamada e revisão da aula anterior
19:20–20:30	Teoria + demonstrações de código no Workbench
20:30–20:45	Intervalo — 15 minutos
20:45–21:40	Exercícios: leve → médio → alto (mão na massa)
21:40–22:00	Correção comentada (gabaritos) e dúvidas

A7

MOTIVAÇÃO

Por que usar JOIN?

- Até aqui, cada consulta acessava apenas uma tabela.
- Com FOREIGN KEY, os dados ficam em tabelas separadas e ligadas.
- O JOIN combina essas tabelas num único resultado.
- **INNER JOIN**
- Só as linhas que existem em AMBAS as tabelas.
- **LEFT JOIN**
- TODAS as linhas da esquerda; sem par, a direita vem NULL.

```
Tabelas de apoio

CREATE TABLE PessoaA (NomeA VARCHAR(30));
INSERT INTO PessoaA VALUES
  ('Joao'), ('Maria'), ('Antonio'),
  ('Samanta'), ('Lucca'), ('Alice');

CREATE TABLE PessoaB (NomeB VARCHAR(30));
INSERT INTO PessoaB VALUES
  ('Flavio'), ('Alice'), ('Joana'),
  ('Mario'), ('Joao'), ('Tereza');
```

A7

JUNÇÕES

INNER JOIN e LEFT JOIN

```
Consulta.sql - MySQL

-- INNER JOIN: nomes presentes nas DUAS tabelas
SELECT A.NomeA, B.NomeB
  FROM PessoaA A
 INNER JOIN PessoaB B
    ON A.NomeA = B.NomeB;
-- Resultado: Joao, Alice

-- LEFT JOIN: todos de A; sem par em B => NULL
SELECT A.NomeA, B.NomeB
  FROM PessoaA A
 LEFT JOIN PessoaB B
    ON A.NomeA = B.NomeB;
```

COMO FUNCIONA

ON

- condição da junção.

Apelidos A, B

- encurtam a escrita.

LEFT

- preserva a esquerda.



EXERCÍCIO PRÁTICO · AULA 7 · NÍVEL LEVE

Primeiro INNER JOIN

CONTEXTO

Uma escola tem Turmas e Alunos ligados por turma_id (da Aula 6).

SUA TAREFA

1. Escreva um INNER JOIN que traga o nome do aluno e o nome da turma.
2. Ordene o resultado pelo nome da turma.
3. Explique com suas palavras o que o ON está comparando.



Primeiro INNER JOIN — solução

```
Solucao.sql - MySQL

SELECT a.nome_aluno, t.nome_turma
  FROM Alunos a
 INNER JOIN Turmas t
    ON a.turma_id = t.turma_id
 ORDER BY t.nome_turma;
```

PONTOS-CHAVE

- ON liga a FK do aluno à PK da turma.
- INNER traz só alunos que têm turma válida.
- Apelidos (a, t) deixam a consulta curta.



EXERCÍCIO PRÁTICO · AULA 7 · NÍVEL MÉDIO

Quem ficou de fora

CONTEXTO

Nem toda turma tem aluno. Você precisa enxergar isso com LEFT JOIN.

SUA TAREFA

1. Insira uma turma nova SEM nenhum aluno vinculado.
2. Faça um LEFT JOIN de Turmas para Alunos mostrando turma e aluno.
3. Observe que a turma vazia aparece com aluno NULL.
4. Depois, liste apenas as turmas sem alunos (WHERE aluno_id IS NULL).



Quem ficou de fora — solução

```
Solucao.sql - MySQL

INSERT INTO Turmas VALUES (4, 'Redes');

SELECT t.nome_turma, a.nome_aluno
  FROM Turmas t
 LEFT JOIN Alunos a ON a.turma_id = t.turma_id;

-- Só as turmas sem alunos
SELECT t.nome_turma
  FROM Turmas t
 LEFT JOIN Alunos a ON a.turma_id = t.turma_id
 WHERE a.aluno_id IS NULL;
```

PONTOS-CHAVE

- LEFT JOIN preserva todas as turmas.
- Sem par, as colunas de Alunos vêm NULL.
- IS NULL isola quem não tem correspondência.



EXERCÍCIO PRÁTICO · AULA 7 · NÍVEL ALTO

Relatório de vendas

CONTEXTO

Uma loja tem Clientes, Produtos e Vendas (com FKs para as duas). Crie as tabelas, insira dados e monte o relatório.

SUA TAREFA

1. Crie Clientes, Produtos e Vendas (IdVenda, IdCliente, IdProduto, PreçoUnitario, Quantidade, DataVenda).
2. Insira ao menos 3 clientes, 3 produtos e 5 vendas.
3. Monte um SELECT com INNER JOINS: cliente, produto, preço, quantidade e o total (preço × quantidade).
4. Ordene pelo IdVenda.



Relatório de vendas — solução

```
Solucao.sql - MySQL

SELECT v.IdVenda,
       c.Nome AS Cliente,
       p.Nome AS Produto,
       v.PrecoUnitario,
       v.Quantidade,
       (v.Quantidade * v.PrecoUnitario) AS ValorTotal,
       v.DataVenda
FROM Vendas v
INNER JOIN Clientes c ON v.IdCliente = c.IdCliente
INNER JOIN Produtos p ON v.IdProduto = p.IdProduto
ORDER BY v.IdVenda;
```

PONTOS-CHAVE

- Um INNER JOIN por tabela relacionada.
- O cálculo $\text{Quantidade} * \text{PrecoUnitario}$ vai no SELECT.
- AS renomeia as colunas do relatório.